Introduction_to_Python

August 11, 2026

1 Introduction to Python

This notebook covers the coding aspects of python programming language.

1.1 A new day in the programming world!

We are in the world of programming. This is where we give instructions to a computer and tell it what to do. How do we give an instruction to a computer? Through a program! We write programs which the computer executes to perform the task we want.

The first thing that we do is say hello to our new world!

[1]:
```
print("Hello World!")
```

```
Hello World!
```

Well, you just said hello!

Here `print()` is a function and inside it is a **string**. The string is enclosed in double quotes `" "`. This is the syntax to say/show something. Print function helps us to output things on the screen.

Let's move on.

Here we will see a comment. A comment helps us to document our code or add comments. A comment does not affect the execution or logic of the code. A comment starts with the `#` symbol.

[2]:
```
# This is a comment
```

Before we move on, there are some important things that we should discuss. Python programs/scripts have the `.py` file extension. The one you are seeing here is a python jupyter notebook. Well no need to worry much. It runs python scripts/programs too. You just have more control running code snippts back and forth and making changes to the code if required without starting to run all the lines of code from scratch.

This notebook has a `.ipynb` file extension.

Another thing which you may come across various cells below, is the use of ***%%python****,* ***%%python3****, etc.* ***Do NOT*** *be worried about them for your course or consider them while writing your code. These are special magic tricks that help each cell to behave as independent program and the variables/functions are not carried over to the next cell. This helps me demostrate each code snippet individually better.*

Let's move on.

Suppose we want the user to give his name to us, and we want to greet him. Lets try to code that.

[7]:
```
name = input("Enter your name: ")
print("Hello", name)
```

```
Hello Alice
```

Woah! That's a lot of code all together. Let us understand step by step.

First, you should understand that each line by itself is a individual statement. This statement executes by itself and has its own logic. We use two lines here:

- The first line uses the `input()` function to ask for an input value from the user. This is called **user input**. The user input is then stored into the `name` variable.
- The second line uses the `print()` function to say Hello with the name. How did the spaces come, whats the formatting logic? We will see them soon.

That's a good start. Let's move onto the next chapters to learn more.

1.2 The Python Propaganda!

Python has become the go-to language for Data Science and AI, not by accident, but because of specific design choices and ecosystem advantages. This chapter explores the "why" behind Python's dominance.

1.2.1 Why Python for Data Science?

Python is preferred in Data Science and AI for several structural reasons:

- Simple, readable syntax that resembles plain English, reducing the learning curve for non-programmers like statisticians and domain experts
- A vast ecosystem of specialized libraries (NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, PyTorch) that handle everything from data manipulation to deep learning
- Strong community support, meaning most problems already have documented solutions or forum discussions
- Interpreted nature, allowing quick testing and iteration without a compile step
- Cross-platform compatibility, so the same code runs on Windows, macOS, and Linux
- Seamless integration with other languages (C, C++, Java) for performance-critical tasks

1.2.2 Python vs Traditional Programming Languages

Aspect	Python	Traditional Languages (C/Java)
Syntax complexity	Minimal, close to pseudocode	Verbose, strict typing rules
Development speed	Fast prototyping	Slower due to boilerplate code
Library support for ML/AI	Extensive and mature	Limited or requires more setup
Community and resources	Massive, active	Smaller in the AI/ML domain
Execution speed	Slower (interpreted)	Faster (compiled)

1.2.3 Role of Python in the AI/ML/Data Science Pipeline

- Data collection and web scraping (using libraries built for handling APIs and files)
- Data cleaning and preprocessing (removing noise, handling missing values)
- Exploratory data analysis (visual and statistical understanding of data)
- Model building and training (implementing ML/DL algorithms)
- Model evaluation and deployment (testing accuracy and pushing models to production)

1.2.4 The Bigger Picture

Python's simplicity does not mean it sacrifices power; its real strength lies in acting as a bridge between mathematical theory and practical implementation. This balance of accessibility and capability is what has made it the unofficial language of modern AI and Data Science.

1.3 Python Basics

Let us come to the core python code and its syntax. Every programming language, just like english language, has its grammar or a ruleset to follow while we write it.

Think of it as a person coming upto you and saying: "School late car was because my I was reach for".

Did you understand what he meant? No, right? There is a specific way he should arrage the words, subject and predicate and use proper tenses so that you are able to understand what he is saying.

In the same manner, programming languages have their own rules to help the computer understand what the program should do. Deviating from that ruleset/syntax breaks the code and the program runs into an error.

That previous text was actually: I was late for school because my car was late.

1.4 Input Output